

**PRAKTIKUM SISTEM CERDAS DAN PENDUKUNG KEPUTUSAN  
SEMESTER GENAP TA 2025/2026**

**LAPORAN PROYEK AKHIR**



**DISUSUN OLEH:**

|                     |   |                                                      |
|---------------------|---|------------------------------------------------------|
| <b>NIM</b>          | : | 123240065<br>123240080                               |
| <b>NAMA</b>         | : | Waladi Lintang Novianto<br>Pande Made Deva Brahmasta |
| <b>PLUG</b>         | : | IF-D                                                 |
| <b>NAMA ASISTEN</b> | : | Hawla Khufyatul Qolbu<br>Akfina Ni'mawati            |

**PROGRAM STUDI INFORMATIKA  
JURUSAN INFORMATIKA  
FAKULTAS TEKNIK INDUSTRI  
UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN"  
YOGYAKARTA  
2026**

## **HALAMAN PENGESAHAN**

## **LAPORAN PROYEK AKHIR**

Disusun oleh:

Waladi Lintang Novianto 123240065

Pande Made Deva Brahmasta 123240080

Telah Diperiksa dan Disetujui oleh Asisten Praktikum Sistem Cerdas dan Pendukung  
Keputusan

Pada Tanggal: .....

**Asisten Praktikum**

**Asisten Praktikum**

**Hawla Khufyatul Qolbu**  
**NIM. 123230021**

**Akfina Ni'mawati**  
**NIM. 1232230076**

## KATA PENGANTAR

Puji syukur saya panjatkan kepada Tuhan Yang Maha Esa yang senantiasa mencurahkan rahmat dan hidayah-Nya sehingga saya dapat menyelesaikan praktikum Sistem Cerdas dan Pendukung Keputusan serta laporan proyek akhir praktikum yang berjudul Sistem Pendukung Keputusan Penentuan Mahasiswa Teladan berdasarkan *Study-Life Balance*. Adapun laporan ini berisi tentang proyek akhir yang saya pilih dari hasil pembelajaran selama praktikum berlangsung.

Tidak lupa ucapan terimakasih kepada asisten praktikum yang selalu membimbing dan mengajari saya dalam melaksanakan praktikum dan dalam menyusun laporan ini. Laporan ini masih sangat jauh dari kesempurnaan, oleh karena itu kritik serta saran yang membangun saya harapkan untuk menyempurnakan laporan akhir ini.

Atas perhatian dari semua pihak yang membantu penulisan ini, saya ucapkan terimakasih. Semoga laporan ini dapat dipergunakan seperlunya.

Yogyakarta, 24 Mei 2026

Penyusun 1,



Waladi Lintang Novianto

NIM. 123240065

Penyusun 2,



Pande Made Deva Brahmasta

NIM. 123240080

## DAFTAR ISI

|                                                     |     |
|-----------------------------------------------------|-----|
| HALAMAN PENGESAHAN .....                            | ii  |
| KATA PENGANTAR.....                                 | iii |
| DAFTAR ISI .....                                    | iv  |
| BAB I PENDAHULUAN .....                             | 1   |
| 1.1 Latar Belakang Masalah .....                    | 1   |
| 1.2 Tujuan Proyek Akhir .....                       | 1   |
| 1.3 Manfaat Proyek Akhir .....                      | 1   |
| BAB II PEMBAHASAN.....                              | 3   |
| 2.1 Dasar Teori .....                               | 3   |
| 2.2 Skenario Kasus & Dataset .....                  | 3   |
| 2.3 Pemodelan Sistem Pendukung Keputusan .....      | 4   |
| 2.4 Perhitungan & Algoritma .....                   | 5   |
| 2.5 Implementasi & Tampilan Sistem .....            | 6   |
| BAB III JADWAL Pengerjaan dan Pembagian Tugas ..... | 29  |
| 3.1 Jadwal Pengerjaan .....                         | 29  |
| 3.2 Pembagian Tugas.....                            | 29  |
| BAB IV KESIMPULAN DAN SARAN.....                    | 30  |
| 4.2 Kesimpulan.....                                 | 30  |
| 4.3 Saran .....                                     | 30  |
| DAFTAR <i>LIBRARY</i> .....                         | 31  |

# BAB I

## PENDAHULUAN

### 1.1 Latar Belakang Masalah

Penilaian mahasiswa teladan di lingkungan akademik pada umumnya masih sangat bergantung pada satu atau dua indikator, misalnya nilai pencapaian akademik (dalam Indeks Prestasi Kumulatif atau *Grade Point Average*) dan keaktifan organisasi (Rektor Universitas Muhammadiyah Kotabumi, 2020). Padahal, mahasiswa yang ideal tidak hanya unggul secara akademis, tetapi juga mampu menjaga keseimbangan hidup (*study-life-balance*). Faktor-faktor penunjang seperti keaktifan berorganisasi, gaya hidup sehat, manajemen waktu digital, dan Kesehatan mental juga tak kalah penting, namun seringkali diabaikan karena sulit diukur secara matematis pasti (Salsa, 2024).

Ketidakpastian dan subjektivitas dalam menilai batas antara "GPA tinggi", "jam olahraga cukup", atau "tingkat stres berlebih" membuat pengambilan keputusan menjadi ambigu. Karena itulah, diperlukan sebuah Sistem Pendukung Keputusan yang mampu merepresentasikan kekaburan penilaian manusia. Proyek akhir ini mengusulkan solusi berupa pembangunan SPK berbasis Logika Fuzzy menggunakan *library* scikit-fuzzy dengan antarmuka web streamlit. Metode Fuzzy dipilih karena kemampuannya memetakan input linguistik secara dinamis untuk menghasilkan skor kelayakan yang tegas sehingga evaluasi mahasiswa teladan dapat dilakukan secara lebih menyeluruh, adil, dan objektif (Prasetya et al., 2025).

### 1.2 Tujuan Proyek Akhir

Tujuan utama dari pembuatan proyek akhir ini adalah:

- 1.1.1 Merancang dan mengimplementasikan Sistem Pendukung Keputusan (SPK) untuk mengevaluasi dan merangking mahasiswa teladan berdasarkan keseimbangan akademik dan gaya hidup.
- 1.1.2 Menerapkan metode Logika Fuzzy menggunakan *library* scikit-fuzzy dengan 5 variabel kriteria (*antecedent*) dan 1 variabel kelayakan (*consequent*).
- 1.1.3 Membuat kode untuk meng-*generate* aturan dinamis yang mampu menghasilkan 162 *rule base* untuk mencegah kekosongan inferensi.
- 1.1.4 Membangun UI berbasis web yang interaktif menggunakan streamlit, lengkap dengan visualisasi data, mekanisme *caching*, dan kalkulator simulasi individu.

### 1.3 Manfaat Proyek Akhir

Adapun manfaat dari pembuatan proyek akhir ini meliputi:

- 1.1.1. Bagi Institusi Pendidikan, sistem ini memberikan alat bantu analisis yang objektif dan terukur bagi pihak rektorat atau program studi dalam proses seleksi kandidat penerima beasiswa atau penghargaan mahasiswa berprestasi, tidak hanya berdasarkan nilai akademik tetapi juga kesehatan fisik dan mental.

- 1.1.2. Bagi Mahasiswa, sistem ini menjadi cerminan evaluasi diri terkait seberapa baik tingkat *work-life-study balance* yang mereka miliki berdasarkan parameter gaya hidup.
- 1.1.3. Bagi Pengembang, sistem ini dapat meningkatkan pemahaman praktis terhadap integrasi model algoritma pendukung keputusan (logika Fuzzy), pemrosesan *dataframe* (Pandas), dan pengembangan aplikasi web (*Streamlit*).

## BAB II PEMBAHASAN

### 2.1 Dasar Teori

Logika kabur (fuzzy logic) adalah suatu pendekatan dalam komputasi yang menangani ketidakpastian dan ketidakjelasan dengan menggunakan nilai keanggotaan yang berkisar antara 0 dan 1, bukan nilai biner konvensional 0 atau 1 (UMA, 2024). Logika fuzzy memungkinkan nilai keanggotaan berada di dalam rentang  $[0, 1]$ . Dalam penerapannya pada Sistem Pendukung Keputusan (SPK), Logika Fuzzy sangat efektif digunakan untuk menyelesaikan masalah *Multi-Criteria Decision Making* yang melibatkan penilaian subjektif manusia.

Proyek ini mengimplementasikan Fuzzy Inference System (FIS) dengan metode Mamdani. Metode Mamdani memerlukan *output* berupa himpunan fuzzy yang kemudian harus diproses melalui tahapan defuzzifikasi untuk menghasilkan nilai *crisp*. Terdapat tiga tahapan utama dalam struktur perhitungan Fuzzy Mamdani:

- a. Fuzzifikasi, yaitu proses mengubah nilai masukan yang bersifat tegas (*crisp input*) dari dataset atau pengguna menjadi nilai fuzzy (derajat keanggotaan) menggunakan fungsi keanggotaan tertentu, seperti kurva segitiga (*trimf*) atau trapesium (*trapmf*).
- b. Inferensi, yaitu proses memetakan nilai input fuzzy menggunakan kumpulan aturan logika berbentuk *IF-THEN* yang merepresentasikan basis pengetahuan sistem. Karena aturan menghubungkan beberapa kriteria menggunakan operator AND, fungsi minimum digunakan untuk menentukan nilai keanggotaan *consequent* dari setiap aturan.
- c. Defuzzifikasi, yaitu proses mengubah kembali hasil inferensi daerah fuzzy menjadi suatu nilai tegas sebagai hasil akhir keputusan. *Library* scikit-fuzzy secara standar menggunakan metode centroid (titik berat), yang menghitung pusat dari area solusi fuzzy untuk mendapatkan nilai akhir.

### 2.2 Skenario Kasus & Dataset

#### 2.2.1. Skenario Kasus

Pihak pengambil keputusan (dalam hal ini pengurus program studi atau rektorat universitas) membutuhkan sebuah mekanisme objektif untuk menentukan predikat "mahasiswa teladan". Kriteria kelayakan tidak lagi hanya diukur melalui performa akademik dan aktif kemahasiswaan, melainkan dievaluasi melalui pemenuhan gaya hidup seimbang (*life-study balance*). Tujuan sistem ini adalah menghasilkan ranking nilai kelayakan mahasiswa, sehingga diperoleh kandidat terbaik yang unggul di bidang akademik, aktif berorganisasi, sehat secara fisik, serta memiliki manajemen waktu dan tingkat stres yang terkontrol secara baik.

#### 2.2.2. Deskripsi Dataset

Dataset yang digunakan dalam proyek ini bersumber dari *platform* Kaggle dengan nama file `global_university_students_performance_habits_10000.csv`. Dataset berformat CSV ini memiliki ukuran total 10.000 baris data dengan 27

kolom atribut yang mencakup informasi mahasiswa dari berbagai belahan negara. Seluruh data pada kolom kriteria utama dipastikan bersih dan tidak mengandung *missing values*.

### 2.2.3. Penerapan Sampling

Untuk menjaga performa pengerjaan komputasi pada antarmuka web Streamlit agar terhindar dari *lag* akibat pembacaan komputasi 10.000 baris, sistem menggunakan data sampel tereduksi sebanyak 1000 baris data. Pengambilan sampel dilakukan secara acak menggunakan *library* Pandas melalui fungsi `pandas.sample(1000, random_state=42)`. Parameter `random_state=42` diset agar hasil penarikan acak data 1000 mahasiswa tersebut bersifat konsisten setiap kali aplikasi berjalan.

## 2.3 Pemodelan Sistem Pendukung Keputusan

Sistem ini memodelkan keseimbangan akademik dan gaya hidup mahasiswa menggunakan 5 variabel input dan 1 variabel output. Berikut adalah penjabaran batasan nilai dan himpunan fuzzy yang digunakan pada masing-masing variabel:

### 2.3.1. Variabel Input

#### a. GPA

- Rentang: 1.9 – 4.0
- Himpunan dan batas nilai (Fungsi *trimf*(segitiga)):
  - Rendah: [1.9, 1.9, 2.8]
  - Sedang: [2.5, 3.2, 3.7]
  - Tinggi: [3.5, 4.0, 4.0]

#### b. Extracurricular Hours

- Rentang (Semesta): 0 – 12 jam/minggu
- Himpunan & Batas Nilai (Fungsi *trimf*(segitiga)):
  - Pasif: [0, 0, 4]
  - Sedang: [2, 6, 9]
  - Aktif: [7, 12, 12]

#### c. Exercise Hours

- Rentang (Semesta): 0 – 10 jam/minggu
- Himpunan & Batas Nilai (Fungsi *trimf*(segitiga)):
  - Kurang: [0, 0, 4]
  - Cukup: [2, 5, 7]
  - Sangat Aktif: [6, 10, 10]

#### d. Screen Time Hours

- Rentang (Semesta): 1 – 12 jam/hari
- Himpunan & Batas Nilai (Fungsi *trapmf*(trapesium)):
  - Ideal: [1, 1, 3, 5]
  - Berlebih: [4, 6, 12, 12]

#### e. Mental Stress Level

- Rentang (Semesta): 1 – 9



- Himpunan & Batas Nilai (Fungsi *trimf* (segitiga)):
  - Rendah: [1, 1, 4]
  - Sedang: [2.5, 5, 7]
  - Tinggi: [6, 9, 9]

### 2.3.2. Variabel Output (Consequent)

Variabel output dalam sistem ini adalah skor teladan yang merepresentasikan nilai kelayakan akhir seorang mahasiswa untuk mendapatkan predikat teladan. Variabel ini memiliki rentang nilai dari 0 hingga 100. Output *crisp* tersebut kemudian dipetakan ke dalam tiga himpunan fuzzy menggunakan fungsi keanggotaan segitiga (*trimf*), yang meliputi kategori kurang dengan parameter batas [0, 0, 40], kategori Layak dengan parameter batas [30, 50, 70], serta kategori Sangat Layak dengan parameter batas [60, 100, 100].

### 2.3.3. Daftar Rule Base

Untuk mencegah masalah kekosongan inferensi yang menghasilkan skor 0, sistem ini menggunakan generasi rule dinamis. Pendekatan ini memetakan seluruh kombinasi matematis ( $3 \times 3 \times 3 \times 2 \times 3 = 162$  kemungkinan) menjadi 162 aturan menggunakan sistem pembobotan internal. Semakin ideal kondisi himpunannya (misal: stres rendah = 3 poin, screen time ideal = 2 poin), semakin besar output akhirnya.

Beberapa sampel aturan yang terbentuk dari 162 *Rule Base* adalah sebagai berikut:

- a. IF (GPA Tinggi) AND (Ekskul Aktif) AND (Olahraga Sangat Aktif) AND (Screen Time Ideal) AND (Stres Rendah) THEN (Skor Teladan Sangat Layak).
- b. IF (GPA Tinggi) AND (Ekskul Sedang) AND (Olahraga Cukup) AND (Screen Time Ideal) AND (Stres Rendah) THEN (Skor Teladan Layak).
- c. IF (GPA Sedang) AND (Ekskul Pasif) AND (Olahraga Kurang) AND (Screen Time Lebih) AND (Stres Tinggi) THEN (Skor Teladan Kurang).
- d. IF (GPA Rendah) AND (Ekskul Pasif) AND (Olahraga Kurang) AND (Screen Time Lebih) AND (Stres Tinggi) THEN (Skor Teladan Kurang).

## 2.4 Perhitungan & Algoritma

Langkah-langkah algoritma Fuzzy Mamdani yang berjalan di *library* scikit-fuzzy pada proyek ini adalah:

- a. Pendefinisian Semesta dan Himpunan  
Menggunakan `np.arange` untuk membuat *array* sumbu-x dengan resolusi 0.05 untuk GPA, dan 0.5 untuk variabel lainnya demi mempercepat proses perhitungan defuzzifikasi.
- b. Fuzzifikasi (Derajat Keanggotaan)  
Sistem menerima angka pasti (contoh: GPA 3.8) dan memetakannya ke dalam kurva segitiga/trapesium untuk mendapatkan nilai  $\mu$  (derajat keanggotaan) antara 0 hingga 1.
- c. Inferensi (Evaluasi Aturan)

Selanjutnya 162 aturan dievaluasi secara serentak. Karena setiap kondisi dalam aturan dihubungkan dengan operator logika AND, algoritma mencari nilai minimum dari seluruh derajat keanggotaan input pada setiap aturan.

d. Agregasi Output

Kurva hasil pemotongan dari seluruh 162 aturan tersebut kemudian digabungkan menjadi satu kurva area wilayah solusi menggunakan fungsi implikasi MAX.

e. Defuzzifikasi

Langkah terakhir adalah mencari titik berat dari kurva agregasi tersebut menggunakan metode *centroid*. Hasil dari perhitungan *centroid* ini mengembalikan *crisp* antara 0-100 yang kemudian digunakan sebagai landasan pemeringkatan data pada *dataframe* Pandas secara *descending*.

## 2.5 Implementasi & Tampilan Sistem

### 2.5.1 Implementasi Logika Fuzzy

Bagian ini merupakan inti pemrosesan dari Sistem Pendukung Keputusan yang dibangun menggunakan library "*skfuzzy*". Implementasi logika Fuzzy Mamdani pada tahap ini mencakup beberapa proses utama, yaitu pembentukan Universe of Discourse untuk lima variabel antecedent (GPA, Ekstrakurikuler, Olahraga, Screen Time, dan Stres) serta satu variabel consequent (Skor Teladan), perancangan fungsi keanggotaan, penyusunan basis aturan (rule base), hingga proses defuzzifikasi menggunakan metode centroid untuk menghasilkan nilai *crisp* akhir.

```
import numpy as np
import skfuzzy as fuzz
import itertools
from skfuzzy import control as ctrl

gpa = ctrl.Antecedent(np.arange(1.9, 4.01, 0.01), 'gpa')
ekskul = ctrl.Antecedent(np.arange(0, 12.5, 0.1), 'ekskul')
olahraga = ctrl.Antecedent(np.arange(0, 10.5, 0.1), 'olahraga')
screen_time = ctrl.Antecedent(np.arange(1, 12.5, 0.1), 'screen_time')
stress = ctrl.Antecedent(np.arange(1, 9.5, 0.1), 'stress')

skor_teladan = ctrl.Consequent(np.arange(0, 101, 1), 'skor_teladan')

# GPA (segitiga): 3 kat
gpa['Rendah'] = fuzz.trimf(gpa.universe, [1.9, 1.9, 2.8])
gpa['Sedang'] = fuzz.trimf(gpa.universe, [2.5, 3.2, 3.7])
gpa['Tinggi'] = fuzz.trimf(gpa.universe, [3.5, 4.0, 4.0])

# Ekskul (segitiga): 3 kat
ekskul['Pasif'] = fuzz.trimf(ekskul.universe, [0, 0, 4])
ekskul['Sedang'] = fuzz.trimf(ekskul.universe, [2, 6, 9])
ekskul['Aktif'] = fuzz.trimf(ekskul.universe, [7, 12, 12])

# Exercise (segitiga): 3 kat
olahraga['Kurang'] = fuzz.trimf(olahraga.universe, [0, 0, 4])
olahraga['Cukup'] = fuzz.trimf(olahraga.universe, [2, 5, 7])
```

```

olahraga['Sangat Aktif'] = fuzz.trimf(olahraga.universe, [6, 10, 10])

# Screen Time (trapesium): 2 kat
screen_time['Ideal'] = fuzz.trapmf(screen_time.universe, [1, 1, 3, 5])
screen_time['Berlebih'] = fuzz.trapmf(screen_time.universe, [4, 6, 12, 12])

# Stress Level (segitiga): 3 kat
stress['Rendah'] = fuzz.trimf(stress.universe, [1, 1, 4])
stress['Sedang'] = fuzz.trimf(stress.universe, [2.5, 5, 7])
stress['Tinggi'] = fuzz.trimf(stress.universe, [6, 9, 9])

skor_teladan['Kurang'] = fuzz.trimf(skor_teladan.universe, [0, 0, 40])
skor_teladan['Layak'] = fuzz.trimf(skor_teladan.universe, [30, 50, 70])
skor_teladan['Sangat Layak'] = fuzz.trimf(skor_teladan.universe, [60, 100, 100])

# rule

bobot_gpa = {'Rendah': 1, 'Sedang': 2, 'Tinggi': 3}
bobot_ekskul = {'Pasif': 1, 'Sedang': 2, 'Aktif': 3}
bobot_olahraga = {'Kurang': 1, 'Cukup': 2, 'Sangat Aktif': 3}
bobot_screen = {'Berlebih': 1, 'Ideal': 2}
bobot_stress = {'Tinggi': 1, 'Sedang': 2, 'Rendah': 3}

rules = []

combination = itertools.product(
    bobot_gpa.keys(),
    bobot_ekskul.keys(),
    bobot_olahraga.keys(),
    bobot_screen.keys(),
    bobot_stress.keys(),
)

for g, e, o, sc, st in combination:
    point_tot = bobot_ekskul[e] + bobot_gpa[g] + bobot_olahraga[o] + bobot_screen[sc] + bobot_stress[st]
    output_kategori=''

    if point_tot >= 11:
        output_kategori = 'Sangat Layak'
    elif 8 <= point_tot <= 10:
        output_kategori = 'Layak'
    else:
        output_kategori = 'Kurang'

    rule = ctrl.Rule(
        gpa[g] & ekskul[e] & screen_time[sc] & stress[st] & olahraga[o], skor_teladan[output_kategori]
    )

    rules.append(rule)

teladan_ctrl = ctrl.ControlSystem(rules)

```

```

teladan_sim = ctrl.ControlSystemSimulation(teladan_ctrl)

def single_inference(gpa_input, ekskul_input, olahraga_input,
screen_time_input, stress_input):
    teladan_sim.input['gpa'] = gpa_input
    teladan_sim.input['ekskul'] = ekskul_input
    teladan_sim.input['olahraga'] = olahraga_input
    teladan_sim.input['screen_time'] = screen_time_input
    teladan_sim.input['stress'] = stress_input

    teladan_sim.compute()

    return teladan_sim.output['skor_teladan']

def rank(df):
    scores = []

    for index, row in df.iterrows():
        try:
            score = single_inference(
                row['GPA'],
                row['extracurricular_hours_per_week'],
                row['exercise_hours_per_week'],
                row['screen_time_hours'],
                row['mental_stress_level']
            )
            scores.append(score)
        except Exception as e:
            scores.append(0)

    df['skor_teladan'] = scores
    df_sorted = df.sort_values(by='skor_teladan',
ascending=False).reset_index(drop=True)
    return df_sorted

```

**Tabel 2.1** Source Code Program Bagian Perhitungan Fuzzy

### 2.5.2 Halaman Utama

Halaman Utama berperan sebagai *landing page* yang memberikan gambaran umum sistem kepada pengguna. Konten yang disajikan meliputi latar belakang pentingnya penilaian keseimbangan hidup mahasiswa di luar aspek akademik, penjelasan singkat mengenai metode Logika Fuzzy Mamdani yang digunakan, serta ringkasan dari dataset global yang menjadi basis sistem, seperti total data mahasiswa, jumlah jurusan, dan rata-rata GPA.

```

import streamlit as st
import pandas as pd
import os

st.set_page_config(
    page_title="SPK Mahasiswa Teladan",
    page_icon="🏠",
    layout="wide",
    initial_sidebar_state="expanded",
)

```

```

@st.cache_data
def load_data():
    BASE_DIR = os.path.dirname(os.path.abspath(__file__))
    file_path = os.path.join(BASE_DIR, "data", "data_sampel.csv")
    df = pd.read_csv(file_path)
    return df.sample(300, random_state=42)

df = load_data()

st.title(":material/school: SPK Pemilihan Mahasiswa Teladan")
st.subheader("Berbasis Logika Fuzzy Mamdani")
st.markdown("----")

st.markdown("""
Penilaian mahasiswa teladan di lingkungan akademik pada umumnya
masih sangat bergantung pada
satu indikator tunggal, yaitu nilai pencapaian akademik (GPA).
Padahal, mahasiswa yang ideal
tidak hanya unggul secara akademis, tetapi juga mampu menjaga
keseimbangan hidup.

Faktor-faktor seperti keaktifan berorganisasi, gaya hidup sehat,
manajemen waktu digital,
dan kesehatan mental juga tak kalah penting, namun seringkali
diabaikan karena sulit diukur
secara matematis pasti.

Sistem ini menggunakan **Logika Fuzzy Mamdani** untuk
merepresentasikan ketidakpastian
penilaian tersebut secara lebih adil dan menyeluruh.
""")

col1, col2= st.columns(2)

with col1:
    with st.container(border=True):
        st.markdown("### :material/bar_chart: Data Explorer")
        st.markdown("""
Eksplorasi dataset mahasiswa secara interaktif.
Lihat distribusi variabel, hubungan antar faktor,
dan filter data sesuai kebutuhan.
""")

with col2:
    with st.container(border=True):
        st.markdown("### :material/calculate: SPK Fuzzy")
        st.markdown("""
Lihat fungsi keanggotaan setiap variabel,
simulasikan skor individu, dan jalankan
perangkingan seluruh dataset.
""")

st.markdown("----")

st.markdown("### Ringkasan Dataset")

col_a, col_b, col_c, col_d = st.columns(4)
col_a.metric("Total Data", len(df))
col_b.metric("Jumlah Jurusan", df["major"].nunique())

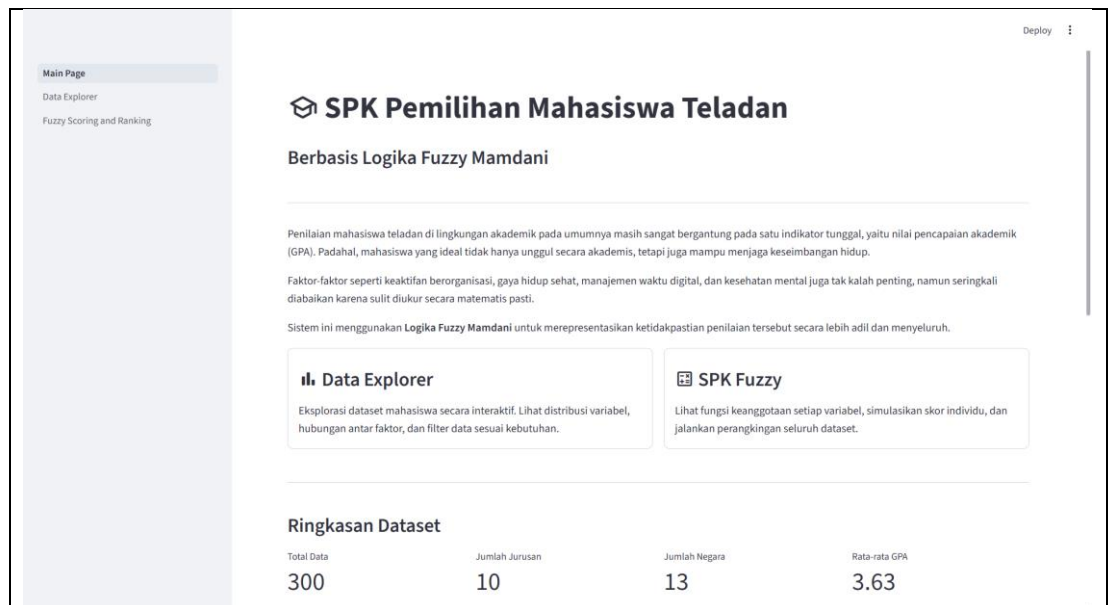
```

```
col_c.metric("Jumlah Negara", df["country"].nunique())
col_d.metric("Rata-rata GPA", f"{df['GPA'].mean():.2f}")

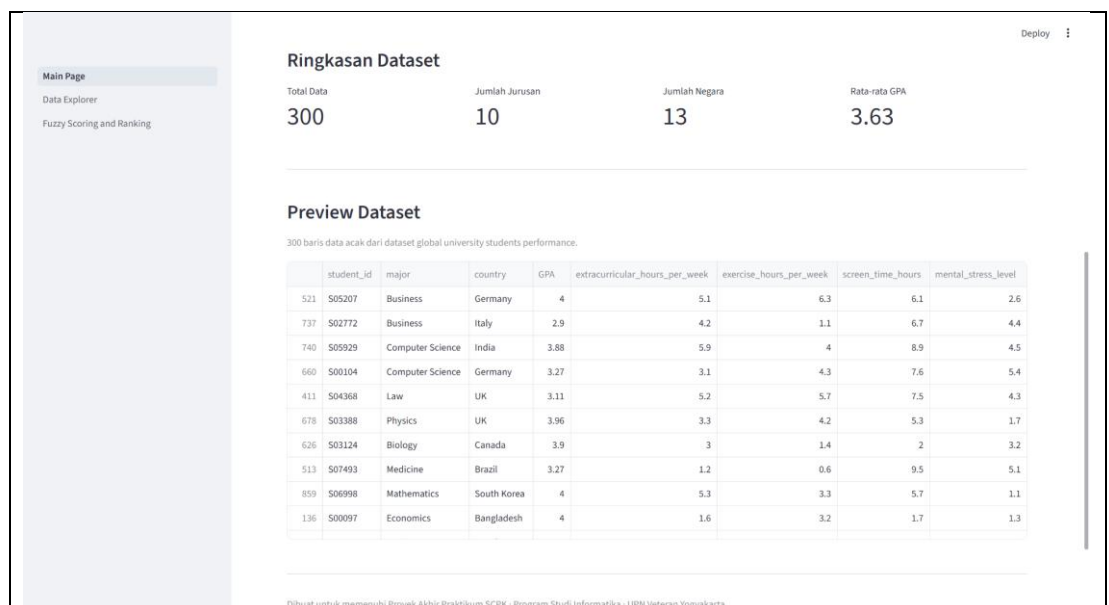
st.markdown("----")
st.markdown("### Preview Dataset")
st.caption("300 baris data acak dari dataset global university students performance.")
st.dataframe(df, use_container_width=True, height=400)

st.markdown("----")
st.caption("Dibuat untuk memenuhi Proyek Akhir Praktikum SCPK · Program Studi Informatika · UPN Veteran Yogyakarta")
```

**Tabel 2.2** *Source Code* Program Bagian Halaman Utama (Main)



**Gambar 2.1** *Interface* Main Page (Ringkasan Metode Fuzzy Mamdani)



**Gambar 2.2** *Interface* Main Page (Ringkasan Dataset)

### 2.5.3 Halaman Eksplorasi Data

Halaman ini menyediakan fitur visualisasi data analitik yang bersifat interaktif, memungkinkan pengguna untuk memfilter dan memahami distribusi dataset sebelum proses perhitungan fuzzy dijalankan. Berbagai grafik statistik tersedia di halaman ini, antara lain korelasi antara GPA dan tingkat stres, distribusi masing-masing variabel input, serta perbandingan rata-rata kondisi mahasiswa antarjurusan. Visualisasi ini sekaligus menjadi dasar argumentasi penggunaan metode fuzzy dalam menghadapi ketidakpastian data.

```
import streamlit as st
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
import os

st.set_page_config(page_title="Data Explorer", layout="wide",
initial_sidebar_state="expanded")

@st.cache_data
def load_data():
    BASE_DIR = os.path.dirname(os.path.abspath(__file__))
    file_path = os.path.join(BASE_DIR, "..", "data",
"data_sampel.csv")
    df = pd.read_csv(file_path)
    return df.sample(300, random_state=42)

df = load_data()

with st.sidebar:
    st.title("Filters")

    selected_majors = st.multiselect(
        "Major",
        options=sorted(df["major"].unique().tolist()),
        default=sorted(df["major"].unique().tolist()),
    )
    selected_countries = st.multiselect(
        "Country",
        options=sorted(df["country"].unique().tolist()),
        default=sorted(df["country"].unique().tolist()),
    )
    gpa_range = st.slider(
        "GPA Range",
        min_value=float(df["GPA"].min()),
        max_value=float(df["GPA"].max()),
        value=(float(df["GPA"].min()), float(df["GPA"].max())),
        step=0.01,
    )

df_filtered = df[
    (df["major"].isin(selected_majors)) &
    (df["country"].isin(selected_countries)) &
    (df["GPA"] >= gpa_range[0]) &
    (df["GPA"] <= gpa_range[1])
]
```

```

]

st.title("Data Explorer")
st.caption(f"Menampilkan {len(df_filtered)} dari {len(df)} data mahasiswa setelah filter diterapkan.")

col1, col2, col3, col4 = st.columns(4)
col1.metric("Total Mahasiswa", len(df_filtered))
col2.metric("Rata-rata GPA", f"{df_filtered['GPA'].mean():.2f}")
col3.metric("Rata-rata Stress", f"{df_filtered['mental_stress_level'].mean():.2f}")
col4.metric("Rata-rata Ekskul", f"{df_filtered['extracurricular_hours_per_week'].mean():.1f} jam/minggu")

tab1, tab2, tab3 = st.tabs(["Overview", "Distribusi", "Eksplorasi per Jurusan"])

with tab1:
    st.subheader("GPA vs Tingkat Stres")
    st.caption(
        "Grafik ini menantang asumsi umum bahwa mahasiswa berprestasi (GPA tinggi) selalu ideal. "
        "Perhatikan mahasiswa dengan GPA tinggi namun tingkat stres yang juga tinggi."
    )

    fig_scatter = px.scatter(
        df_filtered,
        x="GPA",
        y="mental_stress_level",
        color="major",
        hover_data=["student_id", "country", "extracurricular_hours_per_week", "screen_time_hours"],
        labels={
            "GPA": "GPA",
            "mental_stress_level": "Tingkat Stres",
            "major": "Jurusan",
        },
        template="plotly_white",
        opacity=0.75,
    )
    fig_scatter.update_traces(marker=dict(size=7))
    fig_scatter.update_layout(
        height=450,
        legend=dict(orientation="h", yanchor="bottom", y=1.02, xanchor="right", x=1),
    )
    st.plotly_chart(fig_scatter, use_container_width=True)

    st.markdown("---")

    col_a, col_b = st.columns(2)

    with col_a:
        st.subheader("Distribusi GPA")
        st.caption("Sebaran nilai GPA dari seluruh mahasiswa yang difilter.")
        fig_hist = px.histogram(

```



```

        df_filtered,
        x="GPA",
        nbins=25,
        color_discrete_sequence=["#4C78A8"],
        labels={"GPA": "GPA", "count": "Jumlah"},
        template="plotly_white",
    )

    fig_hist.update_layout(height=350, bargap=0.05,
showlegend=False)
    st.plotly_chart(fig_hist, use_container_width=True)

    with col_b:
        st.subheader("Distribusi Tingkat Stres")
        st.caption("Sebaran tingkat stres mahasiswa. Stres tinggi
menunjukkan kondisi yang tidak ideal meski GPA bagus.")
        fig_stress = px.histogram(
            df_filtered,
            x="mental_stress_level",
            nbins=20,
            color_discrete_sequence=["#E45756"],
            labels={"mental_stress_level": "Tingkat Stres", "count":
"Jumlah"},
            template="plotly_white",
        )

        fig_stress.update_layout(height=350, bargap=0.05,
showlegend=False)
        st.plotly_chart(fig_stress, use_container_width=True)

    with tab2:
        st.subheader("Distribusi Variabel")
        st.caption("Pilih variabel untuk melihat sebarannya. Distribusi
yang tidak homogen menjadi alasan penggunaan Fuzzy.")

        var_options = {
            "GPA": "GPA",
            "Ekstrakurikuler (jam/minggu)":
"extracurricular_hours_per_week",
            "Olahraga (jam/minggu)": "exercise_hours_per_week",
            "Screen Time (jam/hari)": "screen_time_hours",
            "Tingkat Stres": "mental_stress_level",
        }

        selected_var_label = st.selectbox("Pilih variabel:",
list(var_options.keys()))
        selected_var = var_options[selected_var_label]

        fig_dist = px.histogram(
            df_filtered,
            x=selected_var,
            nbins=25,
            color_discrete_sequence=["#54A24B"],
            labels={selected_var: selected_var_label, "count": "Jumlah
Mahasiswa"},
            template="plotly_white",
        )

        fig_dist.update_layout(height=400, bargap=0.05,
showlegend=False)
        st.plotly_chart(fig_dist, use_container_width=True)

```

```

with tab3:
    st.subheader("Rata-rata Variabel per Jurusan")
    st.caption(
        "Perbandingan rata-rata setiap variabel antar jurusan. "
        "Membantu melihat apakah ada jurusan yang secara rata-rata
memiliki kondisi tidak ideal."
    )

    var_bar_options = {
        "GPA": "GPA",
        "Ekstrakurikuler (jam/minggu)":
"extracurricular_hours_per_week",
        "Olahraga (jam/minggu)": "exercise_hours_per_week",
        "Screen Time (jam/hari)": "screen_time_hours",
        "Tingkat Stres": "mental_stress_level",
    }

    selected_bar_label = st.selectbox("Pilih variabel:",
list(var_bar_options.keys()), key="bar_var")
    selected_bar = var_bar_options[selected_bar_label]

    avg_per_major = (
        df_filtered.groupby("major")[selected_bar]
        .mean()
        .reset_index()
        .sort_values(selected_bar, ascending=True)
    )

    fig_bar = px.bar(
        avg_per_major,
        x=selected_bar,
        y="major",
        orientation="h",
        color=selected_bar,
        color_continuous_scale="Blues",
        labels={selected_bar: f"Rata-rata {selected_bar_label}"},
        "major": "Jurusan",
        template="plotly_white",
    )

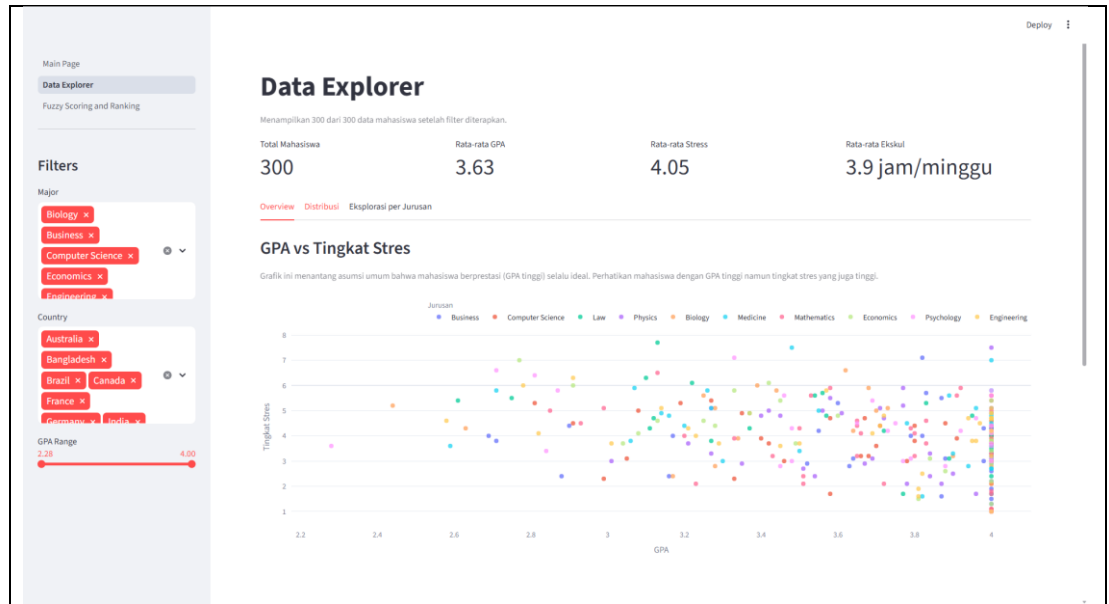
    fig_bar.update_layout(height=420, showlegend=False,
coloraxis_showscale=False)
    st.plotly_chart(fig_bar, use_container_width=True)

    st.markdown("---")

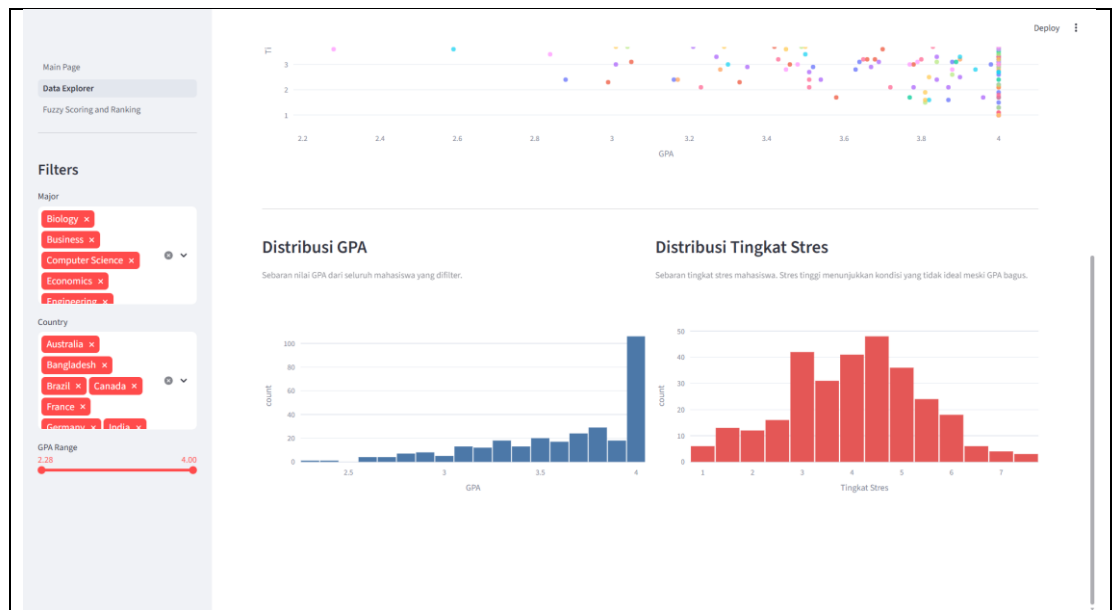
    st.subheader("Raw Data")
    st.caption("Dataset mentah setelah filter diterapkan.")
    st.dataframe(df_filtered.reset_index(drop=True),
use_container_width=True, height=350)

```

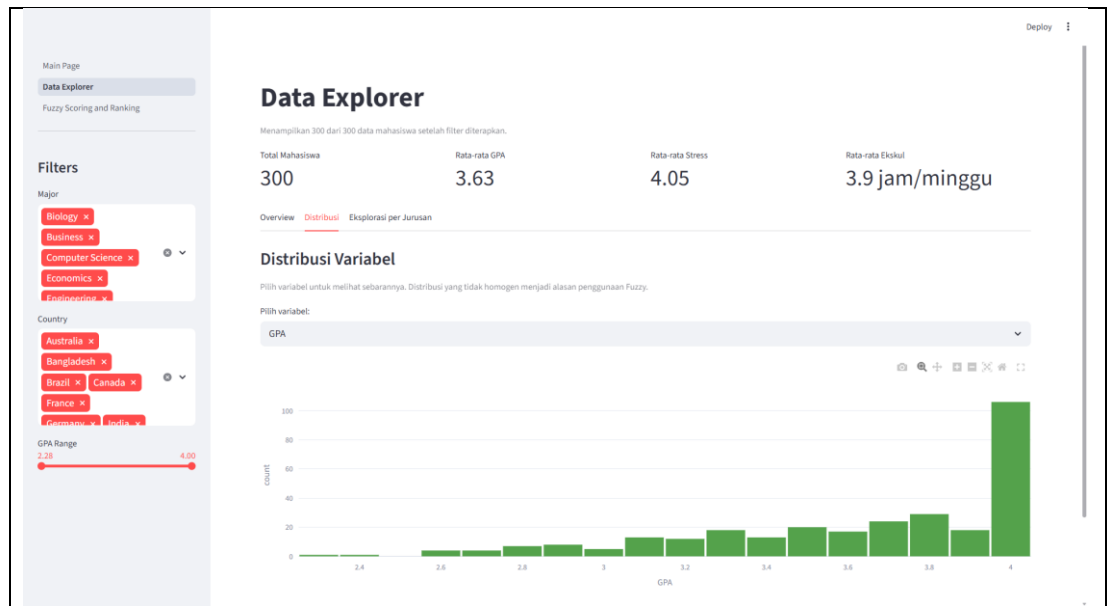
**Tabel 2.3** *Source Code* Program Bagian Halaman Explorasi Data



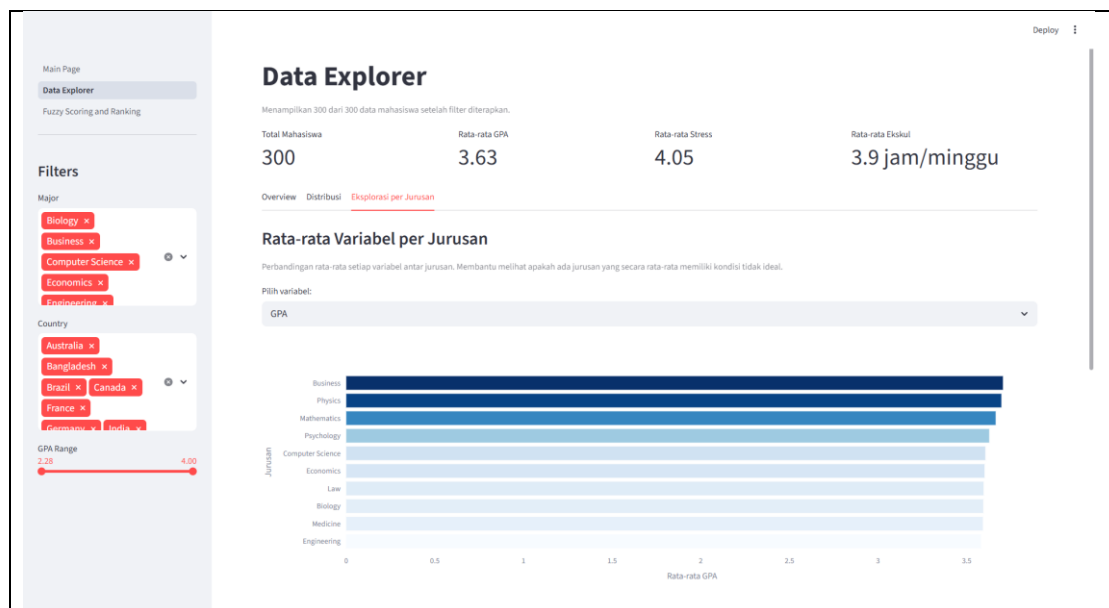
**Gambar 2.3** *Interface* Explorasi Data (Grafik Scatter)



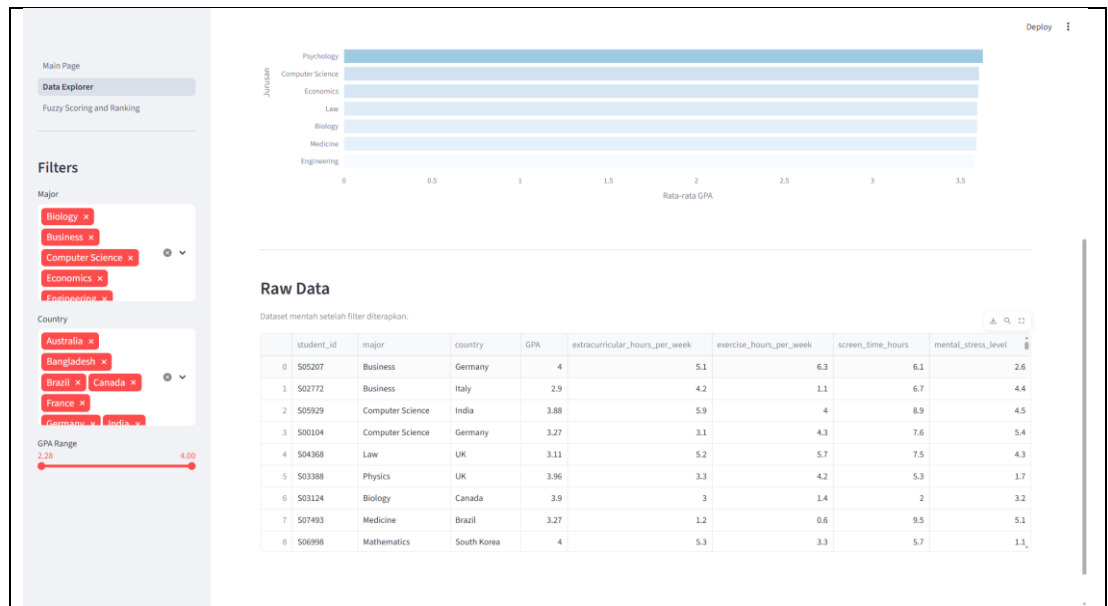
**Gambar 2.4** *Interface* Explorasi Data (Grafik Distribusi GPA & Tingkat Stres)



**Gambar 2.5** Interface Explorasi Data (Distribusi Masing-Masing Variabel)



**Gambar 2.6** Interface Explorasi Data (Explorasi per-Jurusan)



**Gambar 2.7** Interface Explorasi Data (Raw Data)

#### 2.5.4 Halaman SPK Fuzzy Scoring & Ranking

Halaman ini merupakan antarmuka interaktif utama sistem yang memfasilitasi proses pengambilan keputusan secara transparan. Halaman terbagi menjadi tiga tab utama: tab pertama menampilkan visualisasi kurva fungsi keanggotaan untuk memahami batas linguistik tiap variabel, tab kedua menyediakan simulator perhitungan individu dengan slider dinamis yang dilengkapi grafik hasil defuzzifikasi, dan tab ketiga menjalankan proses perangkingan seluruh dataset, menghasilkan tabel urutan mahasiswa dari skor tertinggi hingga terendah.

```
import streamlit as st
import pandas as pd
import numpy as np
import plotly.express as px
import plotly.graph_objects as go
import matplotlib.pyplot as plt
import matplotlib
import os
import sys

matplotlib.use("Agg")

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
sys.path.append(os.path.join(BASE_DIR, ".."))

from fuzzy_engine import rank, single_inference
import skfuzzy as fuzz
from skfuzzy import control as ctrl

st.set_page_config(
    page_title="SPK Fuzzy",
    layout="wide",
    initial_sidebar_state="expanded"
)
```

```

@st.cache_data
def load_data():
    file_path = os.path.join(BASE_DIR, "..", "data",
"data_sampel.csv")
    df = pd.read_csv(file_path)
    return df.sample(300, random_state=42)

@st.cache_data
def run_ranking(df_json):
    df = pd.read_json(df_json)
    return rank(df)

def get_kategori(skor):
    if skor >= 70:
        return "Sangat Layak"
    elif skor >= 40:
        return "Layak"
    else:
        return "Kurang"

def warna_kategori(kategori):
    if kategori == "Sangat Layak":
        return "background-color: #d4edda; color: #155724;"
    elif kategori == "Layak":
        return "background-color: #fff3cd; color: #856404;"
    else:
        return "background-color: #f8d7da; color: #721c24;"

df = load_data()

st.title("SPK Fuzzy Scoring & Ranking")
st.caption("Sistem Pendukung Keputusan Mahasiswa Teladan berbasis
Fuzzy Mamdani.")

tab1, tab2, tab3 = st.tabs(
    ["Fungsi Keanggotaan", "Simulator Individu", "Ranking &
Analisis"]
)

# =====
# TAB 1: FUNGSI KEANGGOTAAN
# =====

with tab1:
    st.subheader("Visualisasi Fungsi Keanggotaan")
    st.caption(
        "Kurva berikut menunjukkan bagaimana sistem Fuzzy Mamdani
mendefinisikan batas linguistik "
        "setiap variabel input. Tidak ada batas tegas, melainkan
derajat keanggotaan (0 sampai 1) "
        "yang merepresentasikan ketidakpastian penilaian manusia."
    )

    mf_configs = [
        {
            "label": "GPA",
            "universe": np.arange(1.9, 4.01, 0.01),
            "sets": {
                "Rendah": ("trimf", [1.9, 1.9, 2.8]),
                "Sedang": ("trimf", [2.5, 3.2, 3.7]),

```

```

        "Tinggi": ("trimf", [3.5, 4.0, 4.0]),
    },
    "colors": {
        "Rendah": ("#E45756", "rgba(228,87,86,0.12)"),
        "Sedang": ("#F58518", "rgba(245,133,24,0.12)"),
        "Tinggi": ("#54A24B", "rgba(84,162,75,0.12)"),
    },
},
{
    "label": "Ekstrakurikuler (jam/minggu)",
    "universe": np.arange(0, 12.5, 0.1),
    "sets": {
        "Pasif": ("trimf", [0, 0, 4]),
        "Sedang": ("trimf", [2, 6, 9]),
        "Aktif": ("trimf", [7, 12, 12]),
    },
    "colors": {
        "Pasif": ("#E45756", "rgba(228,87,86,0.12)"),
        "Sedang": ("#F58518", "rgba(245,133,24,0.12)"),
        "Aktif": ("#54A24B", "rgba(84,162,75,0.12)"),
    },
},
{
    "label": "Olahraga (jam/minggu)",
    "universe": np.arange(0, 10.5, 0.1),
    "sets": {
        "Kurang": ("trimf", [0, 0, 4]),
        "Cukup": ("trimf", [2, 5, 7]),
        "Sangat Aktif": ("trimf", [6, 10, 10]),
    },
    "colors": {
        "Kurang": ("#E45756", "rgba(228,87,86,0.12)"),
        "Cukup": ("#F58518", "rgba(245,133,24,0.12)"),
        "Sangat Aktif": ("#54A24B", "rgba(84,162,75,0.12)"),
    },
},
{
    "label": "Screen Time (jam/hari)",
    "universe": np.arange(1, 12.5, 0.1),
    "sets": {
        "Ideal": ("trapmf", [1, 1, 3, 5]),
        "Berlebih": ("trapmf", [4, 6, 12, 12]),
    },
    "colors": {
        "Ideal": ("#54A24B", "rgba(84,162,75,0.12)"),
        "Berlebih": ("#E45756", "rgba(228,87,86,0.12)"),
    },
},
{
    "label": "Tingkat Stres",
    "universe": np.arange(1, 9.5, 0.1),
    "sets": {
        "Rendah": ("trimf", [1, 1, 4]),
        "Sedang": ("trimf", [2.5, 5, 7]),
        "Tinggi": ("trimf", [6, 9, 9]),
    },
    "colors": {
        "Rendah": ("#54A24B", "rgba(84,162,75,0.12)"),
        "Sedang": ("#F58518", "rgba(245,133,24,0.12)"),

```

```

        "Tinggi": ("#E45756", "rgba(228,87,86,0.12)"),
    },
},
{
    "label": "Skor Teladan (Output)",
    "universe": np.arange(0, 101, 1),
    "sets": {
        "Kurang": ("trimf", [0, 0, 40]),
        "Layak": ("trimf", [30, 50, 70]),
        "Sangat Layak": ("trimf", [60, 100, 100]),
    },
    "colors": {
        "Kurang": ("#E45756", "rgba(228,87,86,0.12)"),
        "Layak": ("#F58518", "rgba(245,133,24,0.12)"),
        "Sangat Layak": ("#54A24B", "rgba(84,162,75,0.12)"),
    },
},
]

col_left, col_right = st.columns(2)

for i, cfg in enumerate(mf_configs):
    fig = go.Figure()

    for set_name, (func_type, params) in cfg["sets"].items():
        universe = cfg["universe"]
        if func_type == "trimf":
            y = fuzz.trimf(universe, params)
        else:
            y = fuzz.trapmf(universe, params)

        line_color, fill_color = cfg["colors"][set_name]

        fig.add_trace(go.Scatter(
            x=universe,
            y=y,
            mode="lines",
            name=set_name,
            line=dict(color=line_color, width=2.5),
            fill="tozero",
            fillcolor=fill_color,
        ))

    fig.update_layout(
        title=dict(text=cfg["label"], font=dict(size=14)),
        xaxis_title=cfg["label"],
        yaxis_title="Derajat Keanggotaan ( $\mu$ )",
        yaxis=dict(range=[0, 1.1]),
        template="plotly_white",
        height=280,
        margin=dict(l=40, r=20, t=50, b=40),
        legend=dict(
            orientation="h", yanchor="bottom", y=1.02,
xanchor="right", x=1
        ),
    )

    if i % 2 == 0:

```



```

        col_left.plotly_chart(fig, use_container_width=True)
    else:
        col_right.plotly_chart(fig, use_container_width=True)

# =====
# TAB 2: SIMULATOR INDIVIDU
# =====
with tab2:
    st.subheader("Simulator Individu")
    st.caption(
        "Masukkan nilai variabel untuk mensimulasikan skor kelayakan
satu mahasiswa."
    )

    col_input, col_output = st.columns([1, 1])

    with col_input:
        gpa_input = st.slider("GPA", min_value=1.9, max_value=4.0,
value=3.5, step=0.01)
        ekskul_input = st.slider(
            "Ekstrakurikuler (jam/minggu)",
            min_value=0.0,
            max_value=12.0,
            value=5.0,
            step=0.1,
        )
        olahraga_input = st.slider(
            "Olahraga (jam/minggu)", min_value=0.0, max_value=10.0,
value=3.0, step=0.1
        )
        screen_input = st.slider(
            "Screen Time (jam/hari)", min_value=1.0, max_value=12.0,
value=5.0, step=0.1
        )
        stress_input = st.slider(
            "Tingkat Stres", min_value=1.0, max_value=9.0,
value=4.0, step=0.1
        )
        hitung = st.button("Hitung Skor", type="primary",
use_container_width=True)

    with col_output:
        if hitung:
            try:
                skor = single_inference(
                    gpa_input, ekskul_input, olahraga_input,
screen_input, stress_input
                )
                kategori = get_kategori(skor)

                st.metric("Skor Teladan", f"{skor:.1f} / 100")

                if kategori == "Sangat Layak":
                    st.success(f"Kategori: {kategori}")
                elif kategori == "Layak":
                    st.warning(f"Kategori: {kategori}")
                else:
                    st.error(f"Kategori: {kategori}")

```

```

st.markdown("***Defuzzifikasi Output (Centroid)**")
st.caption(
    "Area berwarna menunjukkan himpunan output yang
aktif setelah rule-rule dieksekusi. "
    "Garis vertikal adalah titik crisp result dari
metode centroid."
)

universe_out = np.arange(0, 101, 1)
kurang = fuzz.trimf(universe_out, [0, 0, 40])
layak = fuzz.trimf(universe_out, [30, 50, 70])
sangat_layak = fuzz.trimf(universe_out, [60, 100,
100])

    aktivasi_kurang = min(
                                1.0,      max(0.0,
fuzz.interp_membership(universe_out, kurang, skor))
    )
    aktivasi_layak = min(
                                1.0,      max(0.0,
fuzz.interp_membership(universe_out, layak, skor))
    )
    aktivasi_sangat_layak = min(
        1.0,
        max(0.0, fuzz.interp_membership(universe_out,
sangat_layak, skor)),
    )

fig_defuzz = go.Figure()

fig_defuzz.add_trace(
    go.Scatter(
        x=universe_out,
        y=np.minimum(aktivasi_kurang, kurang),
        fill="tozeroy",
        name="Kurang (aktif)",
        line=dict(color="#E45756", width=1.5),
        fillcolor="rgba(228, 87, 86, 0.3)",
    )
)
fig_defuzz.add_trace(
    go.Scatter(
        x=universe_out,
        y=np.minimum(aktivasi_layak, layak),
        fill="tozeroy",
        name="Layak (aktif)",
        line=dict(color="#F58518", width=1.5),
        fillcolor="rgba(245, 133, 24, 0.3)",
    )
)
fig_defuzz.add_trace(
    go.Scatter(
        x=universe_out,
        y=np.minimum(aktivasi_sangat_layak,
sangat_layak),
        fill="tozeroy",
        name="Sangat Layak (aktif)",
        line=dict(color="#54A24B", width=1.5),
        fillcolor="rgba(84, 162, 75, 0.3)",
    )
)

```

```

        )
    )

    fig_defuzz.add_vline(
        x=skor,
        line_dash="dash",
        line_color="#4C78A8",
        line_width=2,
        annotation_text=f"Crisp: {skor:.1f}",
        annotation_position="top right",
    )

    fig_defuzz.update_layout(
        xaxis_title="Skor Teladan",
        yaxis_title="Derajat Keanggotaan ( $\mu$ )",
        yaxis=dict(range=[0, 1.1]),
        template="plotly_white",
        height=320,
        margin=dict(l=40, r=20, t=30, b=40),
        legend=dict(
            orientation="h", yanchor="bottom", y=1.02,
xanchor="right", x=1
        ),
    )

    st.plotly_chart(fig_defuzz,
use_container_width=True)

    except Exception as e:
        st.error(f"Error saat kalkulasi: {e}")
    else:
        st.info("Atur nilai variabel di sebelah kiri, lalu tekan
Hitung Skor.")

# =====
# TAB 3: RANKING & ANALISIS
# =====
with tab3:
    st.subheader("Perangkingan Mahasiswa")
    st.caption(
        "Klik tombol untuk menjalankan proses scoring fuzzy pada
seluruh dataset."
    )

    if st.button("Jalankan Perangkingan", type="primary"):
        with st.spinner("Menghitung skor fuzzy untuk seluruh
mahasiswa..."):
            result = run_ranking(df.to_json())
            result["kategori"] =
result["skor_teladan"].apply(get_kategori)
            st.session_state["ranking_result"] = result

    if "ranking_result" in st.session_state:
        result = st.session_state["ranking_result"]
        result["peringkat"] = range(1, len(result) + 1)

        st.markdown("---")

        c1, c2, c3 = st.columns(3)

```

```

        c1.metric("Sangat Layak", len(result[result["kategori"] ==
"Sangat Layak"]))
        c2.metric("Layak", len(result[result["kategori"] ==
"Layak"]))
        c3.metric("Kurang", len(result[result["kategori"] ==
"Kurang"]))

st.markdown("---")

st.subheader("Tabel Hasil Perangkingan")
display_cols = [
    "peringkat",
    "student_id",
    "major",
    "country",
    "GPA",
    "extracurricular_hours_per_week",
    "exercise_hours_per_week",
    "screen_time_hours",
    "mental_stress_level",
    "skor_teladan",
    "kategori",
]
st.dataframe(
    result[display_cols].reset_index(drop=True),
    use_container_width=True,
    height=400,
)

st.markdown("---")

st.subheader("Top 15 Mahasiswa")
st.caption("Mahasiswa dengan skor teladan tertinggi.")

top15 = result.head(15).copy()
fig_top = px.bar(
    top15,
    x="skor_teladan",
    y="student_id",
    orientation="h",
    color="kategori",
    color_discrete_map={
        "Sangat Layak": "#54A24B",
        "Layak": "#F58518",
        "Kurang": "#E45756",
    },
    hover_data=["major", "country", "GPA",
"mental_stress_level"],
    labels={
        "skor_teladan": "Skor Teladan",
        "student_id": "Student ID",
        "kategori": "Kategori",
    },
    template="plotly_white",
)
fig_top.update_layout(
    height=480,
    yaxis=dict(autorange="reversed"),
    legend=dict(

```

```

orientation="h", yanchor="bottom", y=1.02,
xanchor="right", x=1
    ),
    )
    st.plotly_chart(fig_top, use_container_width=True)

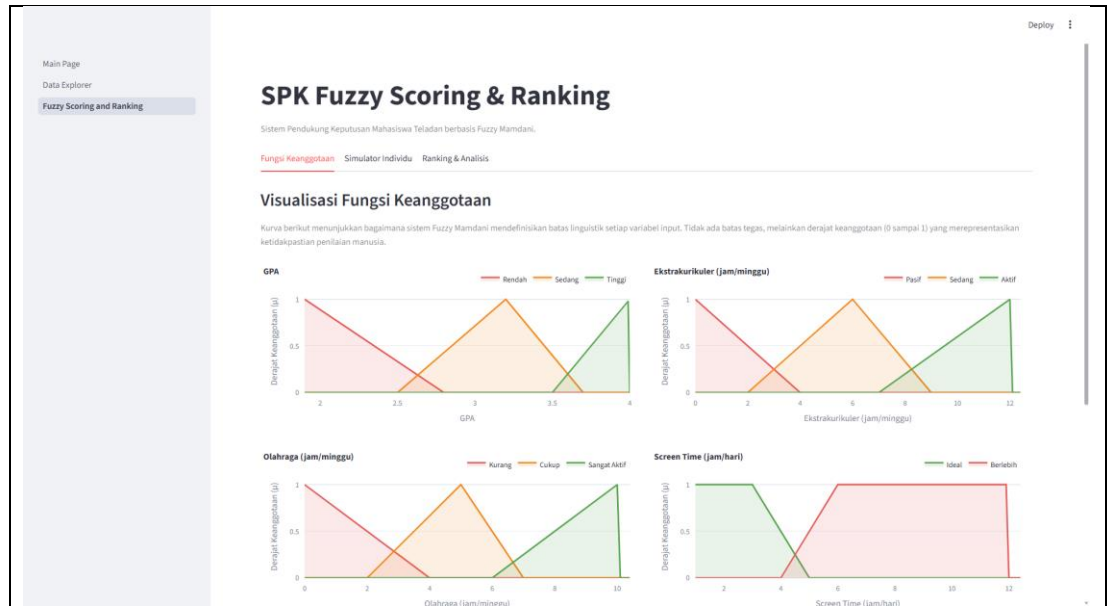
    st.markdown("---")

    st.subheader("GPA vs Skor Teladan")
    st.caption(
        "Grafik ini membuktikan bahwa GPA tinggi tidak otomatis
menghasilkan skor teladan tinggi. "
        "Faktor stres, screen time, olahraga, dan ekstrakurikuler turut
menentukan."
    )

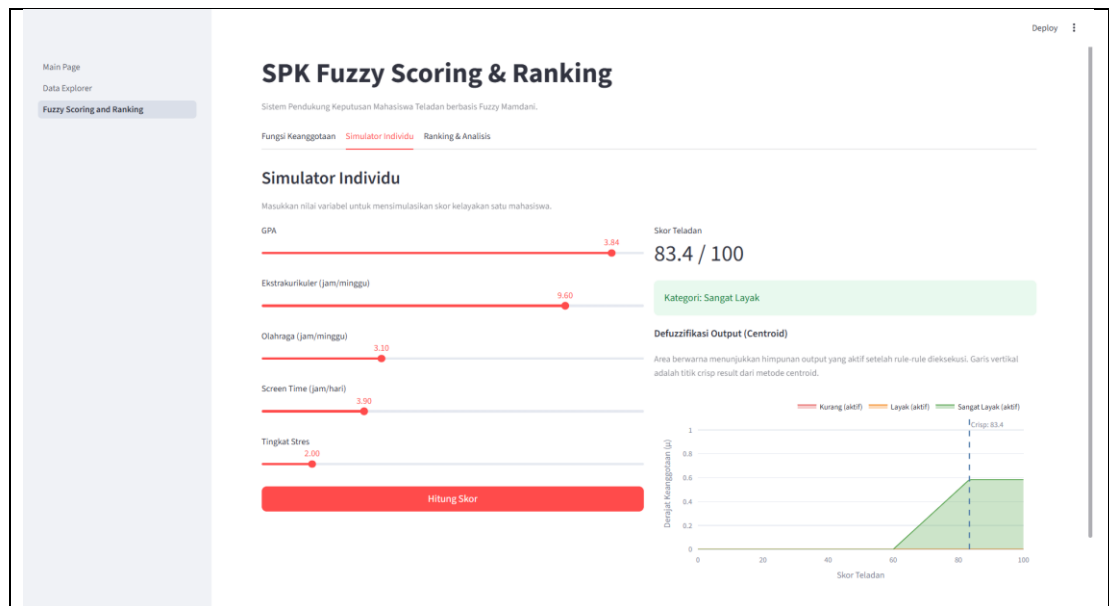
    fig_gpa_skor = px.scatter(
        result,
        x="GPA",
        y="skor_teladan",
        color="kategori",
        color_discrete_map={
            "Sangat Layak": "#54A24B",
            "Layak": "#F58518",
            "Kurang": "#E45756",
        },
        hover_data=[
            "student_id",
            "major",
            "mental_stress_level",
            "screen_time_hours",
        ],
        labels={
            "GPA": "GPA",
            "skor_teladan": "Skor Teladan",
            "kategori": "Kategori",
        },
        template="plotly_white",
        opacity=0.75,
    )
    fig_gpa_skor.update_traces(marker=dict(size=7))
    fig_gpa_skor.update_layout(
        height=450,
        legend=dict(
            orientation="h", yanchor="bottom", y=1.02,
xanchor="right", x=1
        ),
    )
    st.plotly_chart(fig_gpa_skor, use_container_width=True)

```

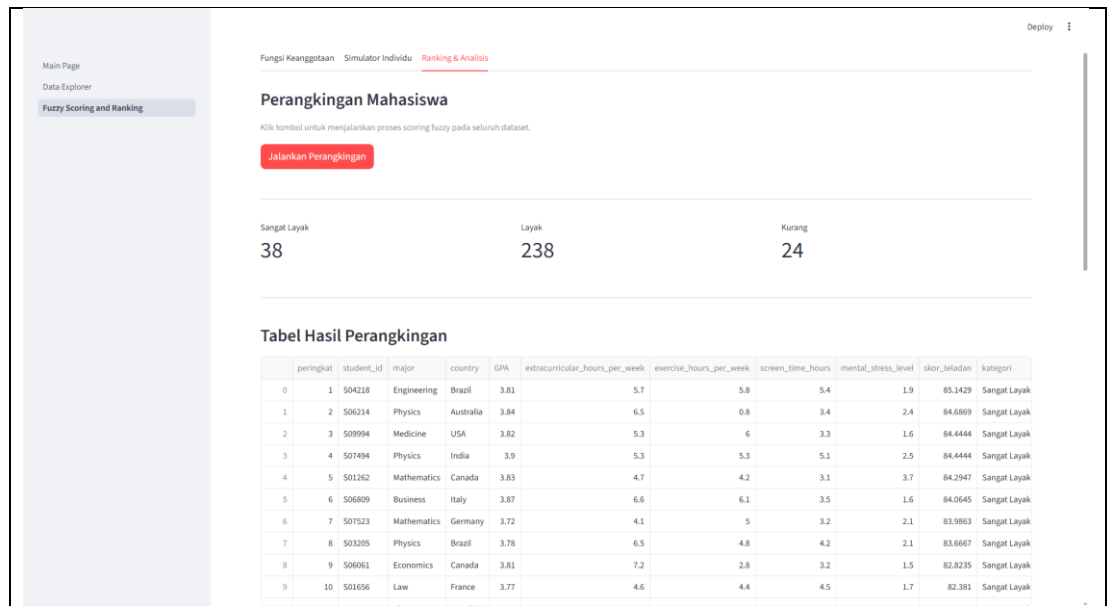
**Tabel 2.4** Source Code Program Bagian Perankingan dan Scoring



**Gambar 2.8** Interface Grafik Fungsi Keanggotaan



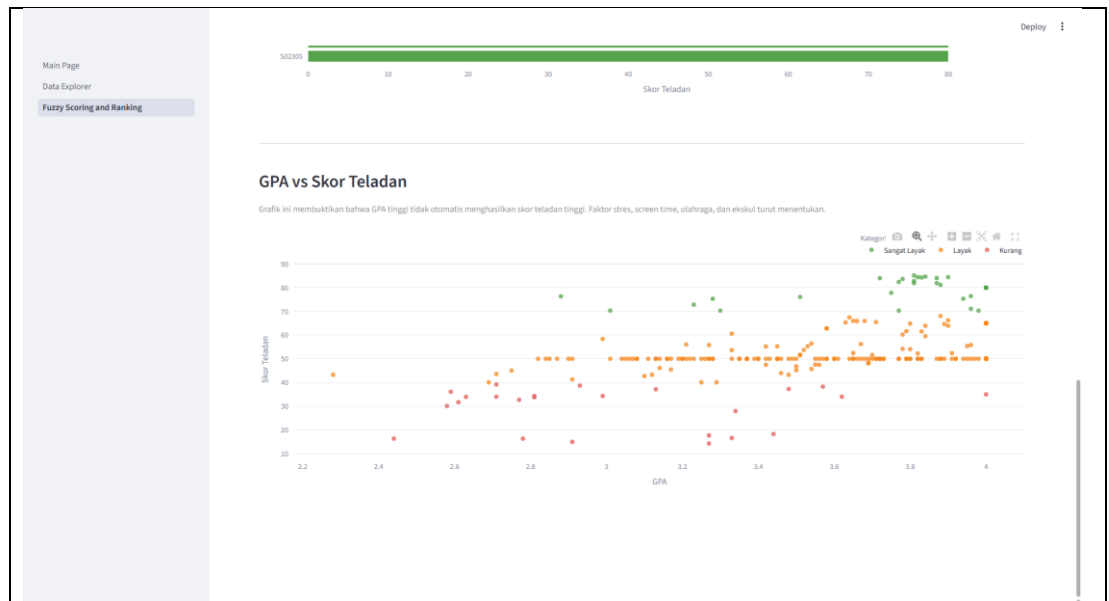
**Gambar 2.9** Interface Simulasi Input Individu dan Skor Teladan



**Gambar 2.10** Interface Perangkingan Mahasiswa



**Gambar 2.11** Interface Top 15 Mahasiswa dengan Kategori Sangat Layak



**Gambar 2.12** *Interface* Perbandingan GPA dan Skor Teladan



### BAB III

### JADWAL Pengerjaan dan Pembagian Tugas

#### 3.1 Jadwal Pengerjaan

| No | Kegiatan                                     | April 2026 |   |   |   | Mei 2026 |   |   |   |
|----|----------------------------------------------|------------|---|---|---|----------|---|---|---|
|    |                                              | Minggu     |   |   |   | Minggu   |   |   |   |
|    |                                              | 1          | 2 | 3 | 4 | 1        | 2 | 3 | 4 |
| 1  | Mencari Ide dan Dataset                      |            |   |   |   |          |   |   |   |
| 2  | Pematangan Ide                               |            |   |   |   |          |   |   |   |
| 3  | Perencanaan Implementasi Teknis              |            |   |   |   |          |   |   |   |
| 4  | Membagi Tugas                                |            |   |   |   |          |   |   |   |
| 5  | Merancang Struktur Proyek                    |            |   |   |   |          |   |   |   |
| 6  | Membuat Logika Fuzzy dan Mekanisme Caching   |            |   |   |   |          |   |   |   |
| 7  | Merancang Tampilan Awal dan Visualisasi Data |            |   |   |   |          |   |   |   |
| 8  | Menyempurnakan Halaman Visualisasi Data      |            |   |   |   |          |   |   |   |
| 9  | Menyempurnakan UI                            |            |   |   |   |          |   |   |   |

#### 3.2 Pembagian Tugas

| No | Nama    | NIM       | Deskripsi Tugas                                                                                                                                                                                                                                                        |
|----|---------|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | Lintang | 123240065 | <ul style="list-style-type: none"> <li>- Mencari Ide dan Dataset</li> <li>- Pematangan Ide</li> <li>- Perencanaan Implementasi Teknis</li> <li>- Membagi Tugas</li> <li>- Merancang Struktur Proyek</li> <li>- Membuat Logika Fuzzy dan Mekanisme Caching</li> </ul>   |
| 2  | Deva    | 123240080 | <ul style="list-style-type: none"> <li>- Mencari Ide dan Dataset</li> <li>- Pematangan Ide</li> <li>- Membagi Tugas</li> <li>- Merancang Tampilan Awal dan Visualisasi Data</li> <li>- Menyempurnakan Halaman Visualisasi Data</li> <li>- Menyempurnakan UI</li> </ul> |

## BAB IV

### KESIMPULAN DAN SARAN

#### 4.2 Kesimpulan

Berdasarkan hasil perancangan dan implementasi Sistem Pendukung Keputusan yang telah dilakukan, dapat disimpulkan bahwa Logika Fuzzy metode Mamdani menggunakan *library* scikit-fuzzy berhasil diimplementasikan untuk merangking kandidat mahasiswa teladan. Penilaian ini terbukti mampu mempertimbangkan faktor keseimbangan gaya hidup meliputi ekstrakurikuler, olahraga, manajemen waktu digital, dan tingkat stres, alih-alih hanya berfokus pada pencapaian akademik. Penggunaan pendekatan *dynamic rule generation* terbukti efektif dalam memetakan 162 *rule base* sehingga sistem mampu menangani seluruh kemungkinan kombinasi data input tanpa mengalami kegagalan inferensi. Selain itu, integrasi model perhitungan analitik ke dalam *interface* web interaktif menggunakan streamlit dapat berjalan dengan lancar. Penerapan fungsi *caching* memori terbukti berhasil mengoptimalkan kinerja komputasi aplikasi saat mengeksekusi iterasi perangkingan terhadap 1000 data sampel secara bersamaan.

#### 4.3 Saran

Pengembangan sistem lebih lanjut dapat difokuskan pada peningkatan skalabilitas dan efisiensi komputasi. Eksplorasi pengujian menggunakan metode Logika Fuzzy Sugeno atau Tsukamoto disarankan sebagai pembanding kinerja, mengingat metode Mamdani dengan defuzzifikasi *centroid* menuntut beban komputasi yang berat untuk pengolahan dataset berskala besar. Skalabilitas sistem juga perlu ditingkatkan dengan mengintegrasikan arsitektur aplikasi web ke *database* sesungguhnya (misalnya SQL atau NoSQL) untuk menggantikan penggunaan dokumen CSV agar proses pembaruan data mahasiswa dapat disediakan secara *real-time*. Penambahan fitur *interface* kriteria interaktif sangat direkomendasikan agar pengambil keputusan dapat mendefinisikan titik parameter kurva himpunan *fuzzy* secara mandiri, sehingga standar penilaian kelayakan dapat disesuaikan dengan kriteria spesifik di masing-masing perguruan tinggi.

## DAFTAR *LIBRARY*

- Prasetya, D., Dwi, A., Putra, R., Saksena, W. A., & Wakhidah, N. (2025). *Penerapan Metode Fuzzy Mamdani dalam Sistem Pendukung Keputusan untuk Evaluasi Kualitas Game Digital Berbasis Multi-Kriteria*.
- Rektor Universitas Muhammadiyah Kotabumi (2020). *Peraturan Penetapan Wisudawan Teladan Universitas Muhammadiyah Kotabumi*.  
<https://baak.umko.ac.id/media/frontend/media/2021/7/peraturan-wisudawan-teladan-umko-9U3eW.pdf>
- Salsa. (2024, September 19). *Memahami Study-Life Balance: Definisi dan Pentingnya bagi Mahasiswa*. <https://Www.Umn.Ac.Id/Memahami-Study-Life-Balance-Definisi-Dan-Pentingnya-Bagi-Mahasiswa/>.
- UMA. (2024). *Logika Kabur: Konsep, Teknik, dan Aplikasinya - Biro Publikasi, Jurnal Ilmiah & Informasi Digital*. <https://Bpjiid.Uma.Ac.Id/2024/06/07/Logika-Kabur-Konsep-Teknik-Dan-Aplikasinya/>.